2.6 Input Validation Annotations, Creating Custom Annotation for Validation

How can we *validate* data or request before sending it to service layer ?

Before sending to service layer means when the data or request is in DTO form.

- In Spring, a *validator* is a component that checks whether Incoming data meets certain *rules* before your application processes it.
- It's used to ensure *data correctness*, *completeness*, and *format* — for example, checking that an email is valid, an age is positive, or a required field isn't missing.

Types of Validation:

1. *Manual Validation*:
   You write your own logic to check values and throw errors if invalid.
   This works, but can become *repetitive* and *messy*.

```
if (employee.getName() == null || employee.getName().isEmpty()) {
    throw new IllegalArgumentException("Name cannot be empty");
}
```

2. *Spring & Jakarta Bean Validation (Recommended)*:
   Uses *annotations* like *@NotNull*, *@Email*, *@Size* to declare validation rules on fields.
   Spring then automatically *checks* them when the request comes in.

***Example*: Validating data using *Jakarta Bean Validation*:**

Step-1 –> Add dependency (if not already present in Spring Boot)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Step-2 –> Add constraints to your *DTO*

```
@Data
public class EmployeeDTO {
    private Long id;
    @NotBlank
    @Size(min = 3, max = 25)
    private String name;
    @Email(message = "Enter valid email")
    private String email;
    @Digits(integer = 6, fraction = 2, message = "Salary must be in format: [XXXXXX.YY]")
    @DecimalMin("9999.99")
    @DecimalMax("200000.50")
    private Double salary;
    @Positive
    @Min(18)
    private Integer age;
    @PastOrPresent
    private LocalDate dateOfJoining;
}
```

Step-3 –> Validate in *Controller*

```
@PostMapping
public ResponseEntity<EmployeeDTO> createNewEmployee(@Valid @RequestBody EmployeeDTO employee) {
    EmployeeDTO employeeDTO = employeeService.createNewEmployee(employee);
    return new ResponseEntity<>(employeeDTO, HttpStatus.CREATED);
}
@PutMapping("/{id}")
public ResponseEntity<EmployeeDTO> replaceEmployeeById(@PathVariable("id") Long id,
                                                        @Valid @RequestBody EmployeeDTO updatedEmployee) {
    return employeeService.replaceEmployeeById(id, updatedEmployee)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
```

- *@Valid* tells Spring to validate *employeeDTO* before calling the *service*.
- If validation fails, Spring returns *400* Bad Request with error details automatically.

Different annotations and works of Jakarta validation

Sl.No.	Annotations	Description
1.	@NotNull	Ensures that the annotated field is not null.
2.	@NotEmpty	Ensures that the annotated field is not null and its size/length is greater than zero. (For collections, arrays, and strings)
3.	@NotBlank	Ensures that the annotated string is not null and its trimmed length is greater than zero.
4.	@Size	Validates that the annotated element's size falls within the specified range.
5.	@Max	Ensures that the annotated element is a number with a value no greater than the specified maximum.
6.	@Email	Validates that the annotated string is a valid email address.
7.	@Pattern	Validates that the annotated string matches the specified regular expression.
8.	@Positive	Ensures that the annotated element is a positive number (greater than zero).
9.	@PositiveOrZero	Ensures that the annotated element is a positive number or zero.
10.	@Negative	Ensures that the annotated element is a negative number (less than zero).
11.	@NegativeOrZero	Ensures that the annotated element is a negative number or zero.
12.	@Past	Ensures that the annotated date or calendar value is in the past.
13.	@PastOrPresent	Ensures that the annotated date or calendar value is in the past or present.
14.	@Future	Ensures that the annotated date or calendar value is in the future.
15.	@FutureOrPresent	Ensures that the annotated date or calendar value is in the future or present.
16.	@Digits	Ensures that the annotated number has up to a specified number of integer and fraction digits.
17.	@DecimalMin	Ensures that the annotated element is a number with a value no less than the specified minimum, allowing for decimal points.
18.	@DecimalMax	Ensures that the annotated element is a number with a value no greater than the specified maximum, allowing for decimal points.
19.	@AssertTrue	Ensures that the annotated boolean field is true.
20.	@AssertFalse	Ensures that the annotated boolean field is false.
21.	@Valid	Validates the associated object recursively (applies bean validation to nested objects).

how to create a custom validation in Spring Boot:

1. *Create a Custom Annotation*
   This annotation is what you'll put on fields in your DTO (e.g., @ValidRRole).

```
@Constraint(validatedBy = RoleValidator.class)
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.PARAMETER, ElementType.FIELD})
public @interface RoleValidation {
   String message() default "Input role must be USER | ADMIN";

   Class<?>[] groups() default {};

   Class<? extends Payload>[] payload() default {};
}
```

1. Class<?>[] groups() default {};
   - This allows you to perform *grouped validations*.
   - Sometimes you want *different validations* in *different contexts* (e.g., one set of rules for CreateUser, another set for UpdateUser).
2. Class<? extends Payload>[] payload() default {};
   - It lets you attach *metadata* to a *constraint violation*.
   - For example, you could attach *severity levels* (INFO, ERROR, CRITICAL) to a *validation error* and later read it in *exception handlers*.

- @Constraint(validatedBy = *RoleValidator.class*) Tells the Bean Validation framework *which validator class* to *run* when it sees this *annotation*.
- @Target({ElementType.*FIELD*, ElementType.*PARAMETER*}) Defines *where* you are *allowed* to *put* this annotation. (e.g. *FIELD*, *PARAMETER*, *METHOD*, *TYPE*)
- @Retention(RetentionPolicy.*RUNTIME*) Defines *when* this annotation is *available* to the *JVM*. (e.g. *SOURCE*, *CLASS*, *RUNTIME*)

2. *Create the Validator*

```
public class RoleValidator implements ConstraintValidator<RoleValidation, String> {
   @Override
   public boolean isValid(String role, ConstraintValidatorContext constraintValidatorContext) {
      Set<String> roles = new HashSet<>(List.of("USER", "ADMIN"));
      return roles.contains(role);
   }
}
```

ConstraintValidator is a **generic interface**.
It connects:

1. Your **custom annotation** (like *@RoleValidation*)
2. The **type of data** you want to validate (like *Strina*)

ConstraintValidator<A, T> is the contract you must implement to connect a ***custom validation annotation A*** with the ***logic*** that ***validates values*** of ***type T***.

3. *Use it in a DTO:*

```
@Data
public class EmployeeDTO {
   @RoleValidation(message = "Employee role must be either USER or ADMIN")
   private String role;
}
```

We can pass our custom *messages* as attribute.

Basically:

1. ***Create annotation*** (@PrimeNumber, @ValidRole)
2. Write ***validator class*** (ConstraintValidator)
3. ***Annotate DTO*** fields with your custom validation
4. Add ***@Valid*** in ***controller*** methods